

3D Steady-State SplinePINNs for solving Navier-Stokes

Sathvik Bhagavan, Ananya Gupta, Tamar Alphaidze
CS-433 Project-2, EPFL, Switzerland

Abstract—Physics-Informed Neural Networks (PINNs) have been widely used for solving physical systems by embedding governing equations into their training. However, limitations such as spectral bias and poor generalization have been observed. To address these challenges, a novel architecture, SteadyStateSplinePINNs, is proposed, building on the work of [1]. Convolutional neural networks (CNNs) are combined with Hermite splines, which are specifically designed for steady-state problems. The model is tested for solving steady-state Navier-Stokes and heat equations in three-dimensional domains and incorporates a physics-informed loss function, boundary condition constraints, and a supervised loss derived from computational fluid dynamics (CFD) data. The representation of smooth and complex solution fields is enhanced through the use of Hermite splines, enabling better capture of the intricate behaviors of steady-state systems. A systematic training pipeline, implemented with Weights and Biases, was employed for experimentation across datasets with varying Reynolds numbers. Notable improvements in solution accuracy were observed, with a reduction in root mean squared error (RMSE) compared to traditional PINNs. The model showed promising results in reconstructing 3D solution fields, effectively capturing key aspects of fluid dynamics and heat transfer problems.

I. INTRODUCTION

Partial Differential Equations (PDEs) are ubiquitous in science and engineering. They govern a wide range of phenomena, from fluid dynamics in airplane design to heat transfer in mobile devices. Despite their prevalence and importance, solving PDEs is often challenging. Many partial differential equations (PDEs) do not have general analytical solutions, and while numerical methods are powerful, they can be computationally expensive and often struggle with high-dimensional problems or complex geometries. This challenge highlights the need for the development of better approaches to solving PDEs. Physics-Informed Neural Networks (PINNs) [2] have emerged as a promising alternative which are computationally efficient, harnessing the power of neural networks to approximate solutions while simultaneously incorporating the governing physical laws. By overcoming some limitations of traditional methods, PINNs hold the potential to transform on solving PDEs across various disciplines.

However, despite their successes, PINNs often suffer from poor training pathologies, and the use of feedforward neural networks can also induce a spectral bias [3] which tends to favor learning low-frequency components over high-frequency ones. Spline PINNs [1] was introduced by Wandel et al. which used Convolution Neural Networks (CNNs) and Hermite Splines whose combinations can be used to express complex solutions with a fine resolution and can generalize to unseen domains.

In this project, a similar architecture is adapted to solve specifically for steady state problems, in this case, the Navier-Stokes equations combined with Heat equation in 3D, referred to as *SteadyStateSplinePINNs*. The Navier-Stokes equations are among the most important PDEs in fluid dynamics, describing how fluids behave under forces such as pressure, viscosity, and external influences. These equations are crucial in the design

and analysis of systems involving fluid flow, including aircraft, automobiles, pipelines, wind turbines, weather forecasting, and even biomedical devices like blood flow pumps.

II. PROBLEM DESCRIPTION

Navier Stokes equations in general form are:

$$\frac{\partial \rho}{\partial t} + \nabla \cdot (\rho \mathbf{u}) = 0 \quad (1)$$

$$\frac{\partial \mathbf{u}}{\partial t} + (\mathbf{u} \cdot \nabla) \mathbf{u} = -\frac{1}{\rho} \nabla p + \frac{\mu}{\rho} \nabla^2 \mathbf{u} + \mathbf{f} \quad (2)$$

where $\mathbf{u}$ is the velocity, ρ is the density, μ is the kinematic viscosity and $\mathbf{f}$ is the external force. Heat equation in general form is:

$$\frac{\partial T}{\partial t} + \mathbf{v} \cdot \nabla T = \alpha \nabla^2 T \quad (3)$$

where T is the temperature, α is the thermal diffusivity. A few assumptions were made regarding the flow to simplify the problem: the fluid was treated as incompressible, it was assumed to be in steady-state and no external forces acting on the system. Boundary conditions included no-slip for velocity and constant temperature at the walls. With these conditions, final equations transform to:

$$\frac{\partial u}{\partial x} + \frac{\partial v}{\partial y} + \frac{\partial w}{\partial z} = 0 \quad (4)$$

$$u \frac{\partial u}{\partial x} + v \frac{\partial u}{\partial y} + w \frac{\partial u}{\partial z} = -\frac{1}{\rho} \frac{\partial p}{\partial x} + \frac{\mu}{\rho} \left(\frac{\partial^2 u}{\partial x^2} + \frac{\partial^2 u}{\partial y^2} + \frac{\partial^2 u}{\partial z^2} \right) \quad (5)$$

$$u \frac{\partial v}{\partial x} + v \frac{\partial v}{\partial y} + w \frac{\partial v}{\partial z} = -\frac{1}{\rho} \frac{\partial p}{\partial y} + \frac{\mu}{\rho} \left(\frac{\partial^2 v}{\partial x^2} + \frac{\partial^2 v}{\partial y^2} + \frac{\partial^2 v}{\partial z^2} \right) \quad (6)$$

$$u \frac{\partial w}{\partial x} + v \frac{\partial w}{\partial y} + w \frac{\partial w}{\partial z} = -\frac{1}{\rho} \frac{\partial p}{\partial z} + \frac{\mu}{\rho} \left(\frac{\partial^2 w}{\partial x^2} + \frac{\partial^2 w}{\partial y^2} + \frac{\partial^2 w}{\partial z^2} \right) \quad (7)$$

$$u \frac{\partial T}{\partial x} + v \frac{\partial T}{\partial y} + w \frac{\partial T}{\partial z} = \alpha \left(\frac{\partial^2 T}{\partial x^2} + \frac{\partial^2 T}{\partial y^2} + \frac{\partial^2 T}{\partial z^2} \right) \quad (8)$$

$$u = 0, \quad v = 0, \quad w = 0 \quad \text{at the wall} \quad (9)$$

$$T = T_0 \quad \text{at the wall} \quad (10)$$

A 3D domain was provided in the form of a CAD file, representing a cuboidal channel with a slight taper and rounded corners. Air entered the channel with a constant mass flow rate and exited at the opposite end. The flow conditions, as described by equations (4)-(10), were assumed to govern the flow. The velocity components u, v, w , pressure p , temperature T are the state variables that need to be learned through training SteadyStateSplinePINNs.

III. MODELS AND METHODS

Training SteadyStateSplinePINNs consists of three main parts - U-Net backbone, Hermite Splines and computing physics losses from the differential equations described above.

A. U-Net

For the neural network component, CNNs is utilized, specifically the U-Net architecture [4] was used based on the insights from previous work [1]. CNNs offer a significant advantage over fully connected Deep Neural Networks (DNNs) in capturing local spatial information. This is particularly important in our case, as the fields under consideration are continuous and exhibit strong similarity between nearby points in space. The U-Net architecture, with its encoder-decoder structure and skip connections, enhances the model's ability to learn spatially varying fields.

To use the model to arbitrary domains with varying geometries, we encoded the domain as a binary mask. In this mask, 1s represent the volume of the domain, while 0s correspond to the boundaries and obstacles present (if any). This binary mask is used as the input to the model. The model then outputs five sets of Hermite spline coefficients, each corresponding to one of five fields as mentioned above.

B. Hermite Splines

Hermite spline are piecewise-defined polynomials that are characterized by their values and their n derivatives at specified support points. By using Hermite splines, it is ensured that the approximated solutions are not only smooth but also capable of accurately representing higher-order derivatives (in this case, 2nd order), which is essential for the solving PDEs correctly.

For the purposes of our model, these support points are arranged on a regular grid, facilitating efficient processing through the CNN framework. Unlike B-spline kernels, which are extended across multiple grid cells, the support of Hermite spline kernels is limited to the immediate neighboring grid cells. This localized support allows for the efficient capture of high-frequency components of the solution while enabling computationally efficient interpolations.

Hermite kernel functions are constructed to map inputs in $[-1, 1]$ to polynomial functions that satisfy the conditions of smoothness and continuity at the support points. The Hermite spline kernel function $h_n^i(x)$ is defined such that the values and derivatives up to the n -th order at the support points (e.g., $x = -1, 0, 1$) are zero, except for the i -th derivative at $x = 0$, which takes a nonzero value to ensure the function resides within the range of $[-1, 1]$. These kernels are then combined linearly to construct continuous piecewise polynomials with $(n + 1)$ -th derivatives of bounded variation.

To extend these kernel functions to 3D dimensional, *tensor product* of 1D Hermite kernel functions is employed. Specifically, a multidimensional kernel $h_{l,m,n}^{i,j,k}(x, y, z)$ can be expressed as:

$$h_{l,m,n}^{i,j,k}(x, y, z) = h_l^i(x)h_m^j(y)h_n^k(z) \quad (11)$$

Here, $h_l^i(x)$, $h_m^j(y)$, and $h_n^k(z)$ represent the 1D Hermite kernel functions in the x , y and z directions respectively.

Using this multidimensional formulation, a continuous Hermite spline approximation $g(x, y, z)$ can be constructed over the grid $(\hat{x}, \hat{y}, \hat{z})$ with corresponding spline coefficients $c_{i,j,k}^{\hat{x}, \hat{y}, \hat{z}}$, as follows:

$$g(x, y, z) = \sum_{i,j,k \in [0:l] \times [0:m] \times [0:n]} \sum_{\hat{x}, \hat{y}, \hat{z}} c_{i,j,k}^{\hat{x}, \hat{y}, \hat{z}} h_{l,m,n}^{i,j,k}(x - \hat{x}, y - \hat{y}, z - \hat{z}). \quad (12)$$

Since the Navier-Stokes equations, which are second-order PDEs, are being solved, the appropriate spline order for our model is determined which is $l = 1$, $m = 1$, and $n = 1$.

This formulation ensures that $g(x, y, z)$ remains smooth and continuous across the domain, with its partial derivatives (e.g., $\partial_x g$, $\partial_y g$, $\partial_z g$) easily computed from the derivatives of the Hermite kernel functions.

The equation (12) can be simplified by considering two 1D kernel functions: h_0 and h_1 . When combined for the 3D case, these result in 8 combinations of 3D kernels, such as:

$$\begin{aligned} h_{0,0,0}(x, y, z) &= h_0(x)h_0(y)h_0(z), \\ h_{0,0,1}(x, y, z) &= h_0(x)h_0(y)h_1(z), \quad \text{and so on.} \end{aligned}$$

Thus, the outer summation can be replaced with an explicit summation over all these combinations for 3D kernels.

The formulation can be further simplified using the property of Hermite kernel functions, which take input in $[-1, 1]$. Outside this, the kernel values are zero. Therefore, for any point of interest (x, y, z) , only the surrounding support points contribute to the field values. These support points correspond to the 8 *grid points* at the vertices of the enclosing cube as shown in fig 3 above.

By replacing $(\hat{x}, \hat{y}, \hat{z})$ with the surrounding grid points, the second summation in the equation is also eliminated, resulting in a more efficient and faster computation of the five fields at the data points.

C. Loss Functions

The loss function is composed of multiple components to incorporate satisfying the differential equations and boundary conditions:

- **Physics-informed loss:** Computing the residuals of the PDEs. This loss ensures that the neural network predictions satisfy the physical equations.
- **Boundary condition loss:** This loss is used as a soft constraint to ensure that the predicted solution adheres to the specified boundary conditions.
- **Supervised loss:** This loss is derived from the error between the predicted values and data. Data was obtained from Computational Fluid Dynamics (CFD) simulations. The supervised loss penalizes deviations between the model's predictions and the provided data points, enabling the model to integrate empirical information effectively.

IV. TRAINING

A. Setup

To train and test the model, two datasets based on different Reynolds numbers were considered:

- $\text{Re} = 1k$: A lower Reynolds number that corresponds to laminar flow.
- $\text{Re} = 100k$: A higher Reynolds number indicative of highly turbulent flow. This value is particularly relevant to our specific problem.

Unet Model had approximately 1.3 million parameters. LBFGS optimizer was employed for training over 100 epochs. For each epoch, 200k points were randomly sampled within the domain for training, and an additional 200k points were used for validation. The supervised loss was computed using 50k randomly chosen supervised data points from 600k points. Finally model was tested on the rest of 550k data points. Given the large volume of data, the model was trained on the Izar HPC cluster equipped with a Nvidia V100 GPU. The training time was about 2 hours.

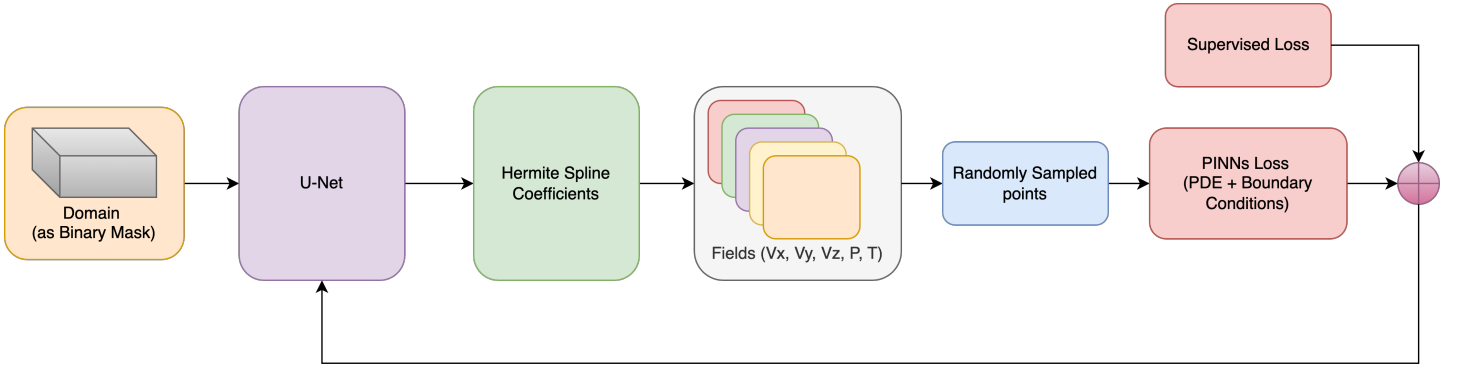


Fig. 1: Pipeline of Spline PINN Model with U-Net, Hermite spline interpolation and PDE Loss

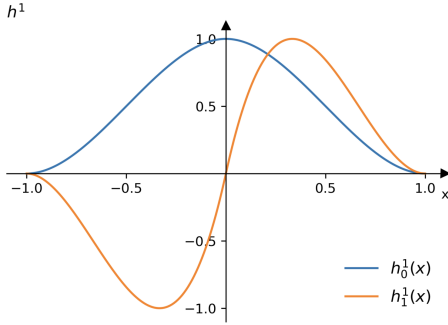


Fig. 2: Hermite spline kernels for 2nd order $n = 1$ in 1D. The kernels are defined on a regular grid and exhibit continuity and smoothness at the support points. Diagram taken from [1]

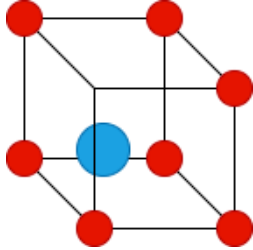


Fig. 3: The field at a specific point in the domain (blue point) is determined only by the surrounding support points (red points). These support points are arranged on a grid for Hermite Splines.

Additionally, a complete end to end machine learning pipeline was setup using Weights and Biases [5] to facilitate ease of running multiple experiments, fine-tune different hyperparameters, and systematically track all results, model weights, and related metrics.

B. Baseline models

PINNs with feed-forward neural network consisting of two hidden layers of 128 neurons each were first implemented as a baseline.

C. SteadyStateSplinePINNs

1) *Without Supervised Loss*: SteadyStateSplinePINNs, as described in the previous section, was first implemented and trained using only physics informed losses on the first configuration. However, the model failed to converge to the true solution fields. It might be because that the solution of the PDEs had very small

pressure gradient. As a result, the velocity fields converged to a *trivial* solution, which is a constant field.

2) *With Supervised Loss*: To address the challenge of avoiding the trivial solution and guiding the model toward the correct solution, a small set of 50k points of training data was incorporated. This dataset was provided to steer the model away from trivial solution and this resulted in significant improvement in the model's performance and the model was able to converge to the true solution fields.

V. RESULTS

Table 1 summarizes the root mean squared error (RMSE) for the baseline PINNs and SteadyStateSplinePINNs. The loss curves are plotted to illustrate the training process, and 3D visualizations of the fields are provided for further insight.

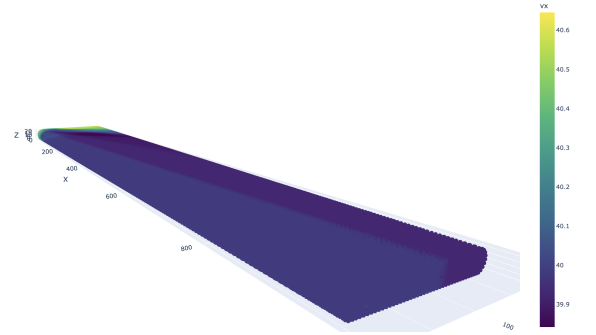


Fig. 4: Plot for PINN

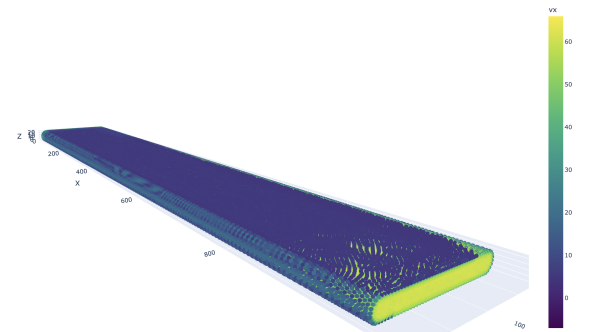


Fig. 5: u Plot for SteadyStateSplinePINN

VI. DISCUSSION

The results clearly show that SteadyStateSplinePINNs outperform baseline PINNs in both configurations. PINNs collapse to

Fields	Range (min, max)	Reynolds=1k				Range (min, max)	Reynolds=100k			
		PINN		SteadyStateSplinePINN			PINN		SteadyStateSplinePINN	
		Training	Test	Training	Test		Training	Test	Training	Test
u (m/s)	(-0.03, 0.82)	0.706	0.705	0.091	0.089	(0.94, 66.74)	19.975	19.965	8.329	8.257
v (m/s)	(-0.24, 0.24)	0.309	0.309	0.007	0.007	(-2.16, 1.81)	0.691	0.691	0.120	0.120
w (m/s)	(-0.44, 1.28)	0.303	0.306	0.008	0.008	(-2.18, 1.78)	0.118	0.117	0.106	0.103
P (Pascal)	(83.4k, 83.5k)	55491.709	55413.910	77.438	75.859	(83.4k, 84.3k)	11401.723	11398.545	9005.992	9004.526
T (K)	(293.15, 338.18)	363.118	364.178	4.016	3.968	(293.69, 338.18)	487.964	487.860	30.677	30.728

TABLE I: Comparison of PINN and SteadyStateSplinePINN RMSE Errors across Reynolds numbers for various fields.

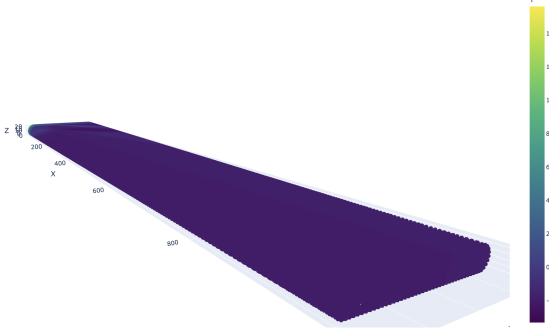


Fig. 6: T Plot for PINN

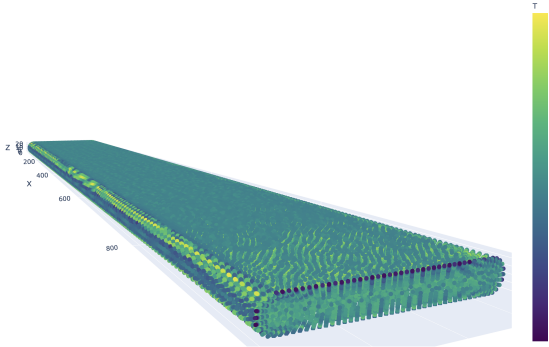


Fig. 7: T Plot for SteadyStateSplinePINN

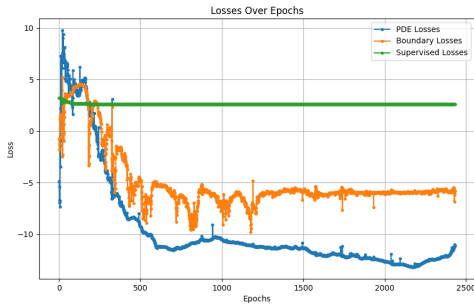


Fig. 8: PINN Losses

the trivial solution as seen in figure 4 and 6 despite the inclusion of data although they perform better in the second configuration. However, they provide very poor and non physical predictions for temperature (as it gives $-200K$) at a Reynolds number of 100k. SteadyStateSplinePINNs, on the other hand, perform well across all states, except for pressure, where the RMSE is slightly higher (~ 9000) in the second configuration. Another observation is that PINNs tend to converge to a very low PDE loss, which is apparent as they often settle on trivial solutions. These findings reveal that CNN-based architectures scale more effectively than feed-

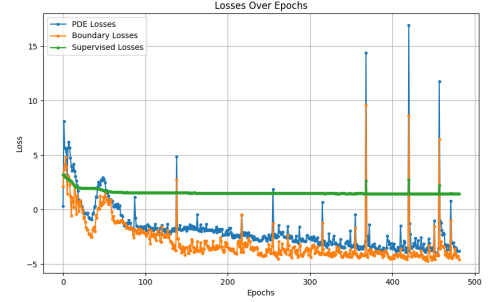


Fig. 9: SteadyStateSplinePINN Losses

forward neural networks. While SteadyStateSplinePINNs show generally strong performance, it is important to note that they achieve good accuracy within the volume but exhibit non-trivial errors on the surface. This might be because of higher gradients near the surface might not be captured accurately.

VII. SUMMARY

SteadyStateSplinePINNs demonstrate the capability to solve the Navier-Stokes equations in 3D domains, including scenarios involving highly turbulent flows. Experimental results highlight their potential as a promising surrogate for numerical flow simulations, offering opportunities for further development and optimization. In this study, SteadyStateSplinePINNs were evaluated at Reynolds numbers up to 100k, with performance compared against baseline PINNs. These types of problems are typically challenging and difficult to train, but the results indicate a feasible and effective approach.

VIII. FUTURE WORK

Several directions for future work have been identified to move closer to the goal of largely or completely replacing the need for intensive CFD simulations with a pre-trained, generalized model.

- 1) Further fine-tuning of the model for higher Reynolds numbers can be done to improve predictions.
- 2) An adaptive weight strategy for the various losses can be implemented to ensure all of them are minimized without overpowering any specific loss term.
- 3) Simple obstacles can be added within the domain to increase complexity, followed by tests featuring more complex obstacle geometries.

IX. ETHICS

To rule out the presence of ethical risks, two categories of stakeholders were evaluated:

- 1) **Direct Stakeholders:** Researchers and developers who designed and implemented the computational model. Their primary responsibility was ensuring that the model was

accurate, reliable, and unbiased. Ethical risks such as introducing errors into the model or misrepresenting the results were mitigated through the following measures:

- *Rigorous Testing*: The model was validated using 200k points per epoch and tested on 550k points, ensuring high reliability and reproducibility of the results.
 - *Systematic Tracking*: A machine learning pipeline using Weights and Biases was implemented to ensure transparency and systematic documentation of hyperparameter tuning, model weights, and performance metrics.
 - *Optimization*: The LBFGS optimizer was employed to ensure efficient convergence, reducing unnecessary computation and energy usage.
- 2) **Indirect Stakeholders**: Broader scientific and engineering communities who may adopt the methods or findings from this study. To address their concerns, the following steps were taken:
- *Open Reporting*: The study results, including data processing techniques, model parameters, and evaluation metrics, were clearly documented to facilitate reproducibility and prevent misuse.
 - *Environmental Impact Assessment*: Given the computational intensity of training on the Izar HPC cluster with a Nvidia V100 GPU, steps were taken to optimize resource usage, such as limiting training to 100 epochs and employing efficient sampling methods (200k training and validation points per epoch). These measures ensured minimal environmental impact while maintaining robust model performance. In addition, SteadyStateSplinePINNs produce a more computationally efficient alternative to CFD simulations, making them more environmentally friendly.

The research did not involve human participants, animals, or sensitive data. Consequently, risks such as privacy violations, exploitation, or harm to vulnerable groups were inherently absent. No further ethical concerns were identified.

REFERENCES

- [1] N. Wandel, M. Weinmann, M. Neidlin, and R. Klein, "Spline-pinn: Approaching pdes without data using fast, physics-informed hermite-spline cnns," 2022. [Online]. Available: <https://arxiv.org/abs/2109.07143>
- [2] M. Raissi, P. Perdikaris, and G. Karniadakis, "Physics-informed neural networks: A deep learning framework for solving forward and inverse problems involving nonlinear partial differential equations," *Journal of Computational Physics*, vol. 378, pp. 686–707, 2019. [Online]. Available: <https://www.sciencedirect.com/science/article/pii/S0021999118307125>
- [3] N. Rahaman, A. Baratin, D. Arpit, F. Draxler, M. Lin, F. A. Hamprecht, Y. Bengio, and A. Courville, "On the spectral bias of neural networks," 2019. [Online]. Available: <https://arxiv.org/abs/1806.08734>
- [4] O. Ronneberger, P. Fischer, and T. Brox, "U-net: Convolutional networks for biomedical image segmentation," in *Medical Image Computing and Computer-Assisted Intervention – MICCAI 2015*, ser. Lecture Notes in Computer Science, N. Navab, J. Hornegger, W. M. Wells, and A. F. Frangi, Eds., vol. 9351. Springer, Cham, 2015, pp. 234–241. [Online]. Available: https://doi.org/10.1007/978-3-319-24574-4_28
- [5] L. Biewald, "Experiment tracking with weights and biases," 2020, software available from wandb.com. [Online]. Available: <https://www.wandb.com/>